

Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39b15ba).

I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. RELATED WORK

Verifier-grounded NetOps evaluation and intent→config generation. Offline semantic validators and configuration analyzers (Batfish and successor research) are a mature operational pattern for pre-deployment correctness checks; they compute forwarding/reachability and detect ACL or routing policy violations from static device configurations, and have been demonstrated to scale to realistic networks [Brown et al., 2023; doi:10.1145/3603269.3604866]. Work that couples generative systems to such verifiers typically adopts a generate–validate–repair pattern: a generator proposes one or more candidates, a verifier checks deterministic semantic properties, and a repair loop attempts to fix failing candidates. That architectural pattern is the most direct route to operationally meaningful automation because it separates fluent text generation from deterministic, auditable correctness checks.

What prior LLM→NetOps evaluations vary on. Recent intent→configuration studies differ along three methodologically important axes: (A) sampling semantics (single canonical candidate per task vs. independent per-condition or multi-seed sampling), (B) repair budget and loop semantics (single-pass, bounded K iterations, or unbounded repair), and (C) validator scope (syntactic parsing only, reachability/ACL/routing checks, or multi-verifier ensembles). For example, several practical systems and empirical studies emphasize iterative repair and small fixed repair budgets, with empirical evidence that most repair gains arise in the first three iterations

(see "Is Three the Magic Number? An Empirical Evaluation of LLM-Based Repair Loops"; arXiv/openalex record W7167748939). Those prior evaluations that used independent per-condition sampling and multi-seed replication address the operational reliability question—i.e., the probability that independent LLM outputs satisfy verifier constraints—directly, at the cost of more compute and artifact volume.

How verifier choice and task synthesis shape measured reliability. Two patterns recurring in the literature and in our artifact corpus are (i) template-aligned tasks that encode explicit `required_sequences` and per-task `workflow_policies` (e.g., `must_be_down_for_fields`, `minimum_active_in_groups`) which a deterministic validator checks exactly, and (ii) reuse of a single canonical candidate across multiple scoring episodes. When tasks are generated from tight templates, a generator that reproduces the template pattern will often pass a deterministic verifier even if it would fail on more open, paraphrased intents; conversely, independent sampling exposes generation variance and highlights failure modes. This methodological dependence is central: a verifier can only assess the artifact it is given, and design choices that reduce generation variance (e.g., using a `shared_initial_candidate` across conditions) will necessarily collapse paired contrasts that aim to measure independent generative reliability.

Artifact-anchored reproducibility as a scientific stance. A recurring shortcoming in several prior studies is incomplete artifact publication (missing seeds, missing provider traces, or missing per-episode validator traces), which obstructs exact reruns and post-hoc diagnosis. Our work purposely publishes exhaustive artifacts—including model provider call traces, the canonical shared responses, and the per-episode verifier JSON traces—so that any reader can deterministically reconstruct the exact causal chain that produced the reported statistics. Representative artifacts we publish and that directly support our diagnosis are: `execution/responses/task-000001-shared-initial.txt` (SHA-256 `8f38c8becd7e1e36...`), the per-episode validator trace `execution/scoring/task-000001-guarded.json` (SHA-256 `0d56c1add7c5a57f...`), and the full execution manifest `execution/execution_manifest.json` which lists every file and SHA-256 used in the run. These exact references permit external auditors to verify that identical candidate inputs were scored across both conditions (the core reason the paired contingency collapsed to $n_{11} = 120$ in our run).

Empirical and methodological contrasts with prior work. Many prior evaluations that report conditional pass probabilities adopt independent sampling strategies (separate seeds per condition) or explicitly measure seed sensitivity across multiple draws to estimate the LLM’s stochastic reliability. Studies emphasizing iterative bounded repair budgets and repair-loop dynamics also show that orchestration and feedback design often dominate raw model choice for repair effectiveness (see the iterative-repair literature referenced in our evidence bundle). By contrast, our preregistered confirmatory design chose `shared_initial_candidate` semantics to control for candidate content while isolating the validator behavior; that design answers a different estimand (validator consistency

under identical inputs) and therefore must be interpreted differently than independent-sampling studies. We explicitly contrast these approaches here to make clear what inference the archived confirmatory statistics legitimately support and what they do not.

Contamination detection and disclosure in the literature. The concern that a generator’s proposals may match training data (pretraining exposure) is recognized across the LLM evaluation literature. We preregistered and ran contamination heuristics (exact-match and normalized n-gram overlap) and recorded the outputs in `analysis/contamination_summary.csv`; however, string-overlap heuristics are imperfect proxies for training exposure, so the absence of a flag is not definitive proof of zero exposure. Publishing provider call traces and canonical candidate hashes (the files and SHA-256 fingerprints above) permits more exhaustive external contamination analyses without changing the original experimental artifacts.

Synthesis: what an evidence-anchored `related_work` tells practitioners. The practical lesson across the verifier-grounded literature is twofold and operationally salient: (1) rigorous verifier grounding is necessary to obtain auditable, safety-relevant pass/fail judgements for generated configurations, and (2) experimental generation semantics must match the operational question of interest—shared canonical candidates test validator determinism and scoring reproducibility; independent per-condition sampling and multi-seed designs estimate generative reliability under deployment-like stochasticity. Our artifact bundle is intentionally complete so that other researchers can replay either paradigm (`shared_initial` or `independent_per_condition`) deterministically and thereby answer the specific operational question they care about without ambiguity.

III. METHODOLOGY

Preregistration and experiment plan. We froze the study prior to observing model outputs: CAND-2026-001-NetConfig-Semantic-v1 (preregistration SHA-256 `7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59`). Primary estimand: `paired_success_rate_difference_guarded_minus_baseline` computed on $N = 120$ paired tasks (40 tasks per topology class: edge, datacenter, multi-AS). Confirmatory generation semantics were preregistered as '`shared_initial_candidate`' (one canonical candidate per task reused for both baseline and guarded episodes). The analysis plan specified `paired_binary_analysis_v1` with exact McNemar test and a 95% bootstrap percentile CI computed with 10,000 resamples (`bootstrap_seed = 1729`). All seeds, analysis code, and CI parameters are archived.

Task synthesis and templates. Tasks were generated programmatically from a deterministic template bank (`generator_id deterministic_netops_task_generator_v5`, version 5.0). Each task manifest entry encodes: `initial_state`, `expected_final_state`, an explicit `required_sequence` (minimal safe command ordering), per-task `workflow_policies` (e.g., `must_be_down_for_fields`, `minimum_active_in_groups`), `allowed_touched_fields`, and a difficulty annotation

(changed_interface_count, transient_rule_count). Templates intentionally encode operational transient constraints (e.g., make-before-break, atomic MTU changes) so validator checks capture semantic and transient safety properties rather than only syntactic validity. Representative manifest entries and their SHA-256s are published (execution/task_manifest.jsonl; example task-000001 manifest SHA-256 ee55625286dcd27c...). Inclusion rule: a task remains in the confirmatory set only if the deterministic template baseline itself parses and the gold template passes the validator unit tests (preregistered). Exclusion and replacement happen prior to model calls to maintain $N = 120$.

Model, orchestration, and call accounting. Declared LLM: gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the exact model configuration artifact is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ff1d82a). The execution contract limited maximum_model_calls = 240; because shared_initial_candidate semantics were used, the run made 120 generation calls (one canonical candidate per task) and recorded provider call traces for reproducibility (execution/provider_calls/*.json). The execution manifest records planned_episode_count = 240 and completed_episode_count = 240; model_calls_used = 120; failed_episode_count = 0. A preregistered retry policy permitted up to two automated retries for infrastructure failures; none occurred.

Generation semantics detail (shared_initial_candidate). The preregistered confirmatory semantics 'shared_initial_candidate' instructs orchestration to generate a single canonical candidate for each task (file execution/responses/task-000XXX-shared-initial.txt) and supply that identical candidate to both scoring episodes (baseline and guarded) for deterministic comparison. This choice intentionally removes generation variance and isolates scoring differences due to scoring condition only. The shared candidate fingerprints are archived (representative: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6c..., task-000003 SHA-256 f7f4db249ecbbce...).

Deterministic validator and scoring rule. Scoring was performed with deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline (archived scoring traces under execution/scoring/*.json) implements: (1) deterministic parsing and command normalization; (2) enforcement of per-task workflow_policies (transient invariants such as minimum_active_in_groups, must_be_down_for_fields); (3) simulation of final_state reasoning (reachability and ACL equivalence checks) in a deterministic semantics engine; (4) normalized configuration canonicalization and violation enumeration. The primary scoring rubric for the confirmatory test was full_intent_constraint_validation: binary pass (score = 1) requires all required_commands present, per-task transient constraints satisfied, and validator final_state consistency with expected_final_state. Each scoring episode emits a JSON trace containing

parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after states, and violation_count. Representative scorer outputs are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Analysis procedures. Primary confirmatory estimator: compute per-task baseline_i and guarded_i binary outcomes (validator pass = 1 / fail = 0) and average the paired differences across $i = 1..120$. Inference: (A) exact McNemar two-sided test on discordant pairs (n_{10}, n_{01}) as preregistered; (B) 95% bootstrap percentile CI on the paired difference using 10,000 resamples (bootstrap_seed = 1729). Missingness treatment: preregistered 'complete_pair_primary' rule—exclude a pair only if both episodes failed for infrastructure reasons; sensitivity analysis treats failed model calls as failures (binary=0). Contamination checks: preregistered heuristics (exact match and normalized n-gram overlap thresholds) were computed and recorded in analysis/contamination_summary.csv; artifact traces in execution/provider_calls and call_cache permit external exposure audits.

Reproducibility and artifact indexing. All artifacts required for exact reproduction are published and SHA-256 hashed in the execution manifest (execution/execution_manifest.json). Key artifact types and representative paths: (1) initial master prompt (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b1597ff1d82a); (2) model configuration (execution/model_configuration.json; SHA-256 a40e96e2...); (3) shared candidates (execution/responses/*-shared-initial.txt) and per-condition responses; (4) per-episode scoring traces (execution/scoring/*.json); (5) provider call traces (execution/provider_calls/*.json) and call cache (execution/call_cache/*) to permit deterministic replay of provider interactions; (6) analysis artifacts (analysis/paired_contingency_table.csv SHA-256 9f25790c7e..., analysis/condition_summary.csv SHA-256 9c77c96f37...). The exhaustive per-file hash manifest is included in execution/execution_manifest.json so a third party can verify byte-level identity and rerun the analysis deterministically.

Pre-specified exploratory analyses and contamination plan. The preregistration included exploratory questions (seed sensitivity, error taxonomy, rank stability via Kendall tau) and a contamination plan: flagging rules for 'suspected_contamination' (exact match OR normalized 5-gram overlap ≥ 0.95) and 'high_overlap' (normalized Levenshtein ≥ 0.9). The primary confirmatory result reports inclusive outcomes; sensitivity analyses excluding or forcing suspected items to failure are recorded as separate artifacts if executed. The artifact bundle includes the outputs of the preregistered heuristics; absence of a flag is not a proof of zero pretraining exposure and is disclosed.

Why the shared_initial design produces a degenerate paired result (diagnostic). Because identical canonical candidates per pair were evaluated by a deterministic validator, any per-pair pass/fail outcome is necessarily identical across the two con-

ditions; deterministic scoring therefore yields n_{11} or n_{00} for each pair. The archived shared candidate files and scoring traces directly document this chain: several representative pairs (e.g., task-000001, task-000002, task-000003) show identical response SHA-256s and identical scoring JSONs across baseline and guarded episodes. The confirmatory contingency table (analysis/paired_contingency_table.csv) is accordingly degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$. The artifact records (execution/responses/*, execution/scoring/*) provide exact, auditable evidence for this causal path.

Runnable, artifact-grounded modifications for an informative comparison. Using the exact published artifacts and code, one can perform deterministic, reproducible reruns that answer independent generative reliability: (A) independent_per_condition confirmatory sampling — generate a separate canonical seed per condition per task (e.g., baseline seed_0, guarded seed_1) and rescore; (B) multi-seed confirmatory design — generate $K \geq 3$ independent seeds per condition and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — add a second deterministic verifier and report intersection/union pass rates to reduce validator–task coupling. These modifications require no new unpublished data and can be implemented deterministically using the archived provider call cache and task generator code paths in the bundle.

IV. RESULTS

Confirmatory outcome and exact, archived counts. All preregistered confirmatory episodes completed and are fully archived in the execution manifest (execution/execution_manifest.json). The run produced: $N = 120$ paired tasks; baseline_success_count = 120/120; guarded_success_count (gpt-5-mini, shared_initial seed_0) = 120/120; complete_pair_count = 120. The paired contingency table is therefore degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$ (analysis/paired_contingency_table.csv; SHA-256 9f25790c7eeafb3562e0710e0cf10b7a47a55ea00fcbcf02ee324f70afef70). Model call accounting recorded model_calls_used = 120 and maximum_model_calls = 240 (execution/execution_manifest.json).

Confirmatory statistical inference (preregistered). Following the frozen analysis plan (paired_binary_analysis_v1) we computed the primary estimator (mean of guarded_i - baseline_i over the 120 pairs) = 0.00. A 10,000-resample percentile bootstrap (seed = 1729 as preregistered) produced a 95% CI [0.00, 0.00]. The exact McNemar two-sided test on the paired 2×2 table yields $p = 1.0$. All analysis artifacts (condition_summary.csv SHA-256 9c77c96f37c4b6ff..., paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7...) are included in the bundle and match these reported values.

Representative validator traces and command counts (artifact grounding). Each scored episode includes a deterministic validator trace showing parsed_command_count, required_command_count, normalized_configuration, and structured before/after final_state. Example: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) records

parsed_command_count = 4, required_command_count = 4, and validation_after.valid = true; the canonical candidate used for the pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Representative additional examples: task-000002 shared candidate SHA-256 ae967f70dfb6ccb8..., task-000003 shared candidate SHA-256 f7f4db249ecbbce.... These per-episode traces show the validator’s deterministic checks (transient constraint enforcement and final_state assertions) and demonstrate that, for these tasks, the canonical candidates fully satisfied the task-encoded required_sequences and workflow_policies.

Why the confirmatory contingency is degenerate — artifact-visible causal chain. The degenerate paired outcome follows directly from two preregistered, archived design choices visible in the bundle: (1) confirmatory_generation_semantics = "shared_initial_candidate" (execution/responses/*-shared-initial.txt) — a single canonical LLM candidate was generated per task and reused verbatim for both baseline and guarded scoring episodes; and (2) task templates encode explicit required_sequences and workflow_policies that are satisfied by canonical, template-aligned candidates. Because the validator is deterministic, identical inputs necessarily yield identical pass/fail outputs. The execution and scoring artifacts (execution/responses/* and execution/scoring/*) provide a precise, auditable record that links the reused canonical candidate to identical validator outcomes across both conditions for each pair.

Contamination checks and reproducibility notes. Preregistered contamination heuristics (exact-match and normalized n-gram overlap) flagged no confirmatory items under the selected thresholds; the recorded contamination summary is archived at analysis/contamination_summary.csv. We explicitly note that absence of a heuristic flag is not a formal proof of zero pretraining exposure. To enable independent audit, all provider call traces and call_cache artifacts are published (execution/provider_calls/*.json, execution/call_cache/*); the full manifest (execution/execution_manifest.json) contains per-file SHA-256 fingerprints to permit byte-for-byte verification (example: execution/model_configuration.json SHA-256 a40e96e212ab87da...).

What these results do and do not show. By construction the confirmatory test correctly answers its preregistered estimand—whether the deterministic verifier treats identical canonical candidates differently under two scoring conditions; the answer is no (paired difference = 0). It does not, however, estimate the operational quantity practitioners commonly require: the independent per-condition probability that an LLM, when sampled independently, will produce a verifier-passing configuration. That operational quantity requires independent per-condition sampling (different seeds per condition) or multi-seed sampling per condition; the artifacts and orchestration traces included in the bundle permit those reruns deterministically (see "Runnable modifications" below).

Runnable, artifact-grounded next steps. The archived pipeline supports immediately reproducible reruns that an-

swer the independent generative reliability question while preserving preregistration and determinism: (A) independent_per_condition confirmatory rerun — generate a distinct seed per condition per task (e.g., baseline seed_0, guarded seed_1), rescore with deterministic_netops_validator_v5, and compute paired contrasts; (B) multi-seed confirmatory design — draw $K \geq 3$ independent seeds per condition per task and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — run a second independent verifier and report intersection/union pass rates to reduce validator–task coupling. Because every element (task generator, model configuration, provider calls, and validator) is archived with SHA-256 fingerprints, these reruns yield deterministic, auditable outputs suitable for confirmatory inference.

V. DISCUSSION

Expanded, artifact-grounded interpretation and actionable diagnostics.

Precise inferential scope. The confirmatory analysis correctly answers the preregistered estimand as written: under shared_initial_candidate semantics, does the validator produce different pass/fail outcomes between two labeled scoring episodes? The archived contingency (analysis/paired_contingency_table.csv SHA-256 9f25790c7eeafb3...) shows a logically inevitable answer of "no difference" because the input artifact was identical per pair. Readers must therefore interpret the reported estimator (paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar $p = 1.0$) as a validation of the execution/analysis pipeline’s determinism under identical inputs—not as evidence that the LLM would produce verifier-valid outputs under independent sampling in realistic deployments.

Artifact evidence chain that produced degeneracy. Three artifact classes in the bundle make the causal chain auditable: (1) canonical candidate files (execution/responses/*-shared-initial.txt; e.g., task-000001 SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6c...), (2) per-episode scoring traces that record parsing and validation outcomes (execution/scoring/*.json; e.g., task-000001-guarded.json SHA-256 0d56c1add7c5a57f...), and (3) the paired contingency and condition summaries (analysis/paired_contingency_table.csv SHA-256 9f25790c..., analysis/condition_summary.csv SHA-256 9c77c96f37c4b6ff...). These three artifact types form a deterministic provenance path: shared_initial candidate → deterministic validator → identical binary score. The bundle contains the exact SHA-256 for each artifact enabling byte-for-byte verification of this chain without re-running the model (execution/execution_manifest.json lists the full manifest).

Why this matters for operational evaluation. Operational decision makers care about independent generative reliability: the probability that an LLM sampled anew (or an LLM ensemble, or an agentic pipeline) will produce a verifier-valid change under realistic prompt/seed/temperature variability and natural-language intent diversity. The current confirmatory

semantics purposely eliminated generation variance by design; the archived artifacts therefore prove the pipeline is reproducible and that the validator correctly accepts the canonical candidates, but they do not estimate the desired deployment quantity (per-task pass probability under independent generation). The distinction is subtle but consequential: reproducible deterministic scoring is necessary for trustworthy pipelines, but it is not sufficient to establish generative robustness.

Runnable, artifact-grounded experiment modifications (preserving reproducibility). Using only the supplied artifacts and configuration files one can deterministically produce informative reruns that estimate independent generative reliability without changing validator code or the template bank: (A) independent_per_condition confirmatory rerun — for each pair generate two independent candidates (one per condition) using distinct seeded calls to the archived generator (the generator id and configuration are archived at execution/model_configuration.json SHA-256 a40e96e2...), then rescore producing a nondegenerate paired table; (B) multi-seed confirmatory design — for each condition draw $K \geq 3$ independent seeds per task and compute per-condition pass probabilities and their bootstrap CIs (this was included as a preregistered exploratory plan); (C) validator diversification — add a second verifier (for example, an independent reachability or ACL analyzer and report intersection and union pass rates) to reduce estimator sensitivity to any single validator’s interpretation of workflow_policies. Each of these modifications is fully implementable using only the published code, seeds, and provider-call traces: because execution/provider_calls/*.json and execution/call_cache/* are included, external auditors can replay, re-seed, and deterministically re-score without needing private provider access to cached model outputs (the call_cache entries themselves contain the exact request/response fingerprints used in the run).

Practical tradeoffs and design guidance. Independent_per_condition sampling increases statistical informativeness but raises computational cost and exposure-management complexity. Multi-seed designs improve precision at the cost of increased model calls; the preregistered plan bounded calls conservatively (planned maximum_model_calls = 240) and specified stopping/retry rules; any rerun should adopt analogous resource bounds. Validator diversification improves construct validity but requires operational alignment: teams must decide whether the union (lenient) or intersection (conservative) of validators is the relevant deployment criterion. Importantly, all of these decisions are traceable and auditable using the provided manifest and SHA-256 list; reproducible governance is feasible by design.

Threats to validity (artifact-identified, expanded). Internal validity: the shared_initial design intentionally removed generation variance, producing a correct but uninformative confirmatory outcome. Construct validity: deterministic task templates and a single validator that enforces template constraints can inflate pass rates for canonical candidates; archived validator traces show that required_sequence checks were

satisfied in all confirmed passes (see `execution/scoring/task-000003-guarded.json` SHA-256 `dad1cf4cfd80b4cc...`). External validity: the experiment used a single declared model (`gpt-5-mini`) and synthetic intent templates; results do not generalize to different LLM families, open natural-language intents, or production topologies. Conclusion validity: the degenerate contingency yields trivial hypothesis test outputs; interpreting $p = 1.0$ as evidence of equal operational reliability would be a category error. All these threats are documented and evidenced in the artifact bundle, enabling third parties to quantify them by rerunning the alternative designs described above.

Operational implications and recommended forensic checklist. For practitioners evaluating LLM→NetOps pipelines we offer an artifact-based checklist grounded in this study’s materials: (1) preregister the estimand and generation semantics (shared vs independent) and archive both; (2) publish canonical candidate files, provider call traces, and validator traces with SHA-256s to enable third-party audit; (3) when estimating generative robustness, use `independent_per_condition` sampling and multi-seed replication; (4) require validator diversification for high-stakes rollouts; (5) perform contamination/exposure checks over published candidates and provider traces (the bundle includes `analysis/contamination_summary.csv` for replication). Our run demonstrates that following these steps makes both positive and null outcomes auditable and actionable.

Concluding interpretive note. The experiment as executed is scientifically honest, reproducible, and valuable: it demonstrates a fully-articulated pipeline and identifies a methodological pitfall that can silently convert an intended independent test into a deterministic sameness check. Because all artifacts and SHA-256 fingerprints (including the immutable initial master prompt `provenance/master_prompt.txt` SHA-256 `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15` and the preregistration SHA-256 `7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fde671a17`) are published, teams can (and should) deterministically transform this reproducible baseline into an informative assessment of independent generative reliability via the minimal, artifact-grounded reruns described above.

VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered

exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.

- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to `gpt-5-mini` under the recorded configuration and `deterministic_netops_validator_v5`; they do not generalize across model families, agentic pipelines, or other validators.

VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (`execution/execution_manifest.json` and `analysis/*`). Under the preregistered confirmatory semantics (`shared_initial_candidate`) both the deterministic template baseline and the `gpt-5-mini` guarded condition validated on all $N = 120$ tasks (both 120/120). Archived evidence (`shared_initial` candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (`independent_per_condition` sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution artifacts—including the immutable master prompt reference (`provenance/master_prompt.txt`; SHA-256 `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15`) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: `declared_model_version = gpt-5-mini` accessed via provider adapter '`openai_responses`' and adapter family '`hosted_netops_gvr_v1`' (`execution/model_configuration.json` SHA-256 `a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82`). Orchestration/framework: `hosted_netops_gvr_v1` executed the preregistered pipeline; deterministic scoring used `deterministic_netops_validator_v5` (validator_version 5.0).

Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebd71a37.

Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>